

CODE+OUTPUT OF LAB TASK EXAMPLE:

colab.research.google.com/drive/1kWlg-JpypalEDOOYqSagWVPdmsyu6EZ#scrollTo=F-yv6hBUnfhE

Set Google Chrome as your default browser and pin it to your taskbar

Untitled0.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[1] import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pandas.plotting import scatter_matrix
```

```
[2] data = {
    "StudentID": ["S01", "S02", "S03", "S04", "S05", "S06", "S07", "S08",
                  "S09", "S10", "S11", "S12", "S13", "S14", "S15"],
    "StudyHours": [2, 5, 7, 3, 8, 10, 12, np.nan, 6, 9, -1, 4, 11, 18, 5],
    "Attendance": [60, 72, 85, 68, 78, 88, 92, 80, 75, 84, 90, 65, 95, 90, 72],
    "AssignmentScore": [40, 55, 70, 48, 65, 80, 85, 76, 60, 73, 82, 52, 88, 91, 57],
    "ExamScore": [45, 63, 81, 50, 38, 74, 88, 79, 61, 72, 84, 58, 90, 96, 64],
    "StudyGroup": ["Low", "Low", "High", "Low", "Medium", "Medium",
                  "High", "High", "Medium", "Medium", "high", "Low", "High", "High", "Medium"],
    "PassStatus": ["Fail", "Pass", "Pass", "Pass", "Fail", "Pass", "Pass",
                  "pass", "Pass", "Pass", "Pass", "Pass", "Pass", "Pass", "Pass"]}

```

Activate Windows
Go to Settings to activate Windows.

4:07 PM Python 3

Untitled0.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[2] raw_df = pd.concat([raw_df, raw_df.iloc[[8]]], ignore_index=True)
raw_df.head(10)
```

	StudentID	StudyHours	Attendance	AssignmentScore	ExamScore	StudyGroup	PassStatus
0	S01	2.0	60	40	45	Low	Fail
1	S02	5.0	72	55	63	Low	Pass
2	S03	7.0	85	70	81	High	Pass
3	S04	3.0	68	48	50	Low	Pass
4	S05	8.0	78	65	38	Medium	Fail
5	S06	10.0	88	80	74	Medium	Pass
6	S07	12.0	92	85	88	High	Pass
7	S08	NaN	80	76	79	High	pass
8	S09	6.0	75	60	61	Medium	Pass
9	S10	9.0	84	73	72	Medium	Pass

Activate Windows
Go to Settings to activate Windows.

4:07 PM Python 3

File Edit View Insert Running Tools Help

Commands + Code + Text Run all

RAM Disk

```
[3] print("Shape of the dataset (rows, columns):", raw_df.shape)
print()
raw_df.head()
```

Shape of the dataset (rows, columns): (16, 7)

	StudentID	StudyHours	Attendance	AssignmentScore	ExamScore	StudyGroup	PassStatus
0	S01	2.0	60	40	45	Low	Fail
1	S02	5.0	72	55	63	Low	Pass
2	S03	7.0	85	70	81	High	Pass
3	S04	3.0	68	48	50	Low	Pass
4	S05	8.0	78	65	38	Medium	Fail

```
[4] raw_df.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16 entries, 0 to 15
Data columns (total 7 columns):
Column Non-Null Count Dtype

0 StudentID 16 non-null object
1 StudyHours 15 non-null float64
2 Attendance 16 non-null int64
3 AssignmentScore 16 non-null int64
4 ExamScore 16 non-null int64
5 StudyGroup 16 non-null object
6 PassStatus 16 non-null object
dtypes: float64(1), int64(3), object(3)
memory usage: 1.0+ KB

```
[5] variable_types = pd.DataFrame({
    "Variable": raw_df.columns,
    "Suggested Type": [
        "ID",
        "Numerical",
        "Numerical",
        "Numerical",
        "Numerical",
        "Categorical",
        "Categorical"
    ]
})
variable_types
```

	Variable	Suggested Type
0	StudentID	ID
1	StudyHours	Numerical
2	Attendance	Numerical

Activate Windows


```
[9]
✓ 0s
invalid_studyhours = raw_df[raw_df["StudyHours"] < 0]
invalid_attendance = raw_df[(raw_df["Attendance"] < 0) | (raw_df["Attendance"] > 100)]
invalid_exam = raw_df[(raw_df["ExamScore"] < 0) | (raw_df["ExamScore"] > 100)]

print("Rows with invalid StudyHours:")
display(invalid_studyhours)
print("Rows with invalid Attendance:")
display(invalid_attendance)
print("Rows with invalid ExamScore:")
display(invalid_exam)
```

▼ ... Rows with invalid StudyHours:

	StudentID	StudyHours	Attendance	AssignmentScore	ExamScore	StudyGroup	PassStatus
10	S11	-1.0	90	82	84	high	Pass

Rows with invalid Attendance:

	StudentID	StudyHours	Attendance	AssignmentScore	ExamScore	StudyGroup	PassStatus
--	-----------	------------	------------	-----------------	-----------	------------	------------

Rows with invalid ExamScore:

	StudentID	StudyHours	Attendance	AssignmentScore	ExamScore	StudyGroup	PassStatus
--	-----------	------------	------------	-----------------	-----------	------------	------------

Activate Windows

▼ ... Rows with invalid Attendance:

	StudentID	StudyHours	Attendance	AssignmentScore	ExamScore	StudyGroup	PassStatus
--	-----------	------------	------------	-----------------	-----------	------------	------------

Rows with invalid ExamScore:

	StudentID	StudyHours	Attendance	AssignmentScore	ExamScore	StudyGroup	PassStatus
--	-----------	------------	------------	-----------------	-----------	------------	------------

Activate Windows

```
[10]
✓ 0s
plt.figure(figsize=(7, 4))
plt.boxplot(raw_df["StudyHours"].dropna())
plt.title("Box Plot of StudyHours (Raw Data)")
plt.ylabel("StudyHours")
plt.show()
```

▼ ...

A box plot titled "Box Plot of StudyHours (Raw Data)". The y-axis is labeled "StudyHours" and ranges from 2.5 to 17.5. The plot shows a median around 6.5, a first quartile (Q1) around 4.5, and a third quartile (Q3) around 9.5. There are whiskers extending from approximately 1.5 to 12.0. A single outlier is present at approximately 17.5.

Activate Windows
Go to Settings to activate Windows.

▼ ...

A box plot titled "Box Plot of StudyHours (Raw Data)". The y-axis is labeled "StudyHours" and ranges from 0.0 to 12.5. The plot shows a median around 6.5, a first quartile (Q1) around 4.5, and a third quartile (Q3) around 9.5. There are whiskers extending from approximately 1.5 to 12.0. A single outlier is present at approximately 17.5.

Activate Windows

```
[11] clean_df = raw_df.copy()
# Remove extra spaces and standardize category labels
clean_df["StudyGroup"] = clean_df["StudyGroup"].str.strip().str.title()
clean_df["PassStatus"] = clean_df["PassStatus"].str.strip().str.title()
# Replace invalid negative StudyHours with NaN
clean_df.loc[clean_df["StudyHours"] < 0, "StudyHours"] = np.nan
# Fill missing StudyHours with median
median_studyhours = clean_df["StudyHours"].median()
clean_df["StudyHours"] = clean_df["StudyHours"].fillna(median_studyhours)
# Remove duplicate rows
clean_df = clean_df.drop_duplicates()
print("Median used to fill missing StudyHours:", median_studyhours)
print("Shape after cleaning:", clean_df.shape)
clean_df
```

Median used to fill missing StudyHours: 6.5
Shape after cleaning: (15, 7)

	StudentID	StudyHours	Attendance	AssignmentScore	ExamScore	StudyGroup	PassStatus
0	S01	2.0	60	40	45	Low	Fail
1	S02	5.0	72	55	63	Low	Pass

Activate Windows
Go to Settings to activate Windows.

Commands + Code + Text Run all

3	S04	3.0	68	48	50	Low	Pass
4	S05	8.0	78	65	38	Medium	Fail
5	S06	10.0	88	80	74	Medium	Pass
6	S07	12.0	92	85	88	High	Pass
7	S08	6.5	80	76	79	High	Pass
8	S09	6.0	75	60	61	Medium	Pass
9	S10	9.0	84	73	72	Medium	Pass
10	S11	6.5	90	82	84	High	Pass
11	S12	4.0	65	52	58	Low	Pass
12	S13	11.0	95	88	90	High	Pass
13	S14	18.0	98	91	96	High	Pass
14	S15	5.0	72	57	64	Medium	Pass

```
[12] print("Missing values after cleaning:")
display(clean_df.isnull().sum())
print("Duplicate rows after cleaning:", clean_df.duplicated().sum())
print()
print("Unique StudyGroup values after cleaning:")
print(clean_df["StudyGroup"].unique())
print()
print("Unique PassStatus values after cleaning:")
print(clean_df["PassStatus"].unique())
```

Missing values after cleaning:

StudentID	0
StudyHours	0
Attendance	0
AssignmentScore	0
ExamScore	0
StudyGroup	0

Activate Windows
Go to Settings to activate Windows.

StudyGroup

0

PassStatus

0

dtype: int64

Duplicate rows after cleaning: 0

Unique StudyGroup values after cleaning:
['Low' 'High' 'Medium']

Unique PassStatus values after cleaning:
['Fail' 'Pass']

[13]

0s

pass_counts = clean_df["PassStatus"].value_counts()

pass_percent = clean_df["PassStatus"].value_counts(normalize=True) * 100

pass_summary = pd.DataFrame({

"Frequency": pass_counts,

"Percentage": pass_percent.round(2)

})

print(pass_summary)

plt.figure(figsize=(6, 4))

plt.bar(pass_counts.index, pass_counts.values)

plt.title("Pass Status Count")

plt.xlabel("PassStatus")

plt.ylabel("Number of Students")

plt.show()

Frequency

Percentage

PassStatus

Pass

Fail

13

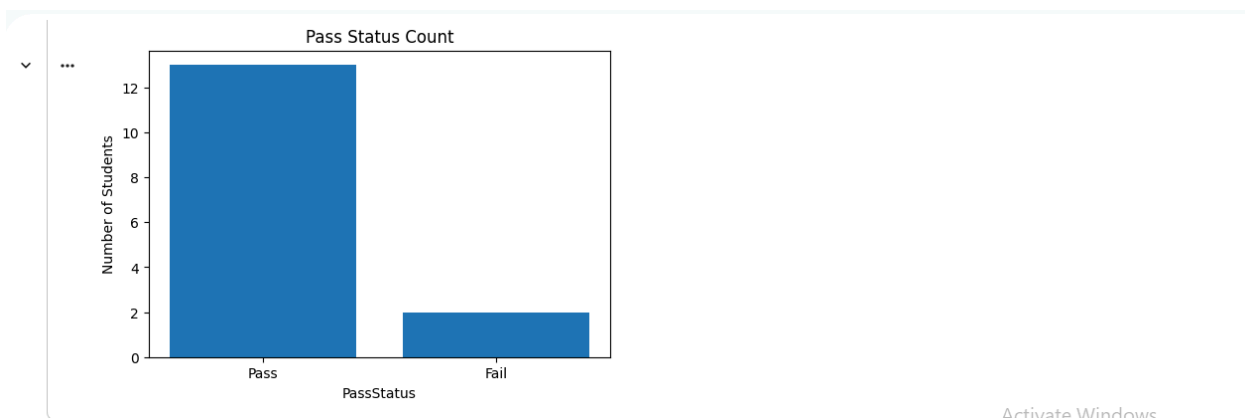
2

86.67

13.33

Pass Status Count

Activate Windows



[14]

exam_summary = clean_df["ExamScore"].describe()

print(exam_summary)

Histogram

plt.figure(figsize=(7, 4))

plt.hist(clean_df["ExamScore"], bins=6, edgecolor="black")

plt.title("Histogram of ExamScore")

plt.xlabel("ExamScore")

plt.ylabel("Frequency")

plt.show()

Box Plot

plt.figure(figsize=(7, 4))

plt.boxplot(clean_df["ExamScore"])

plt.title("Box Plot of ExamScore")

plt.ylabel("ExamScore")

plt.show()

count

mean

std

min

75%

15.000000

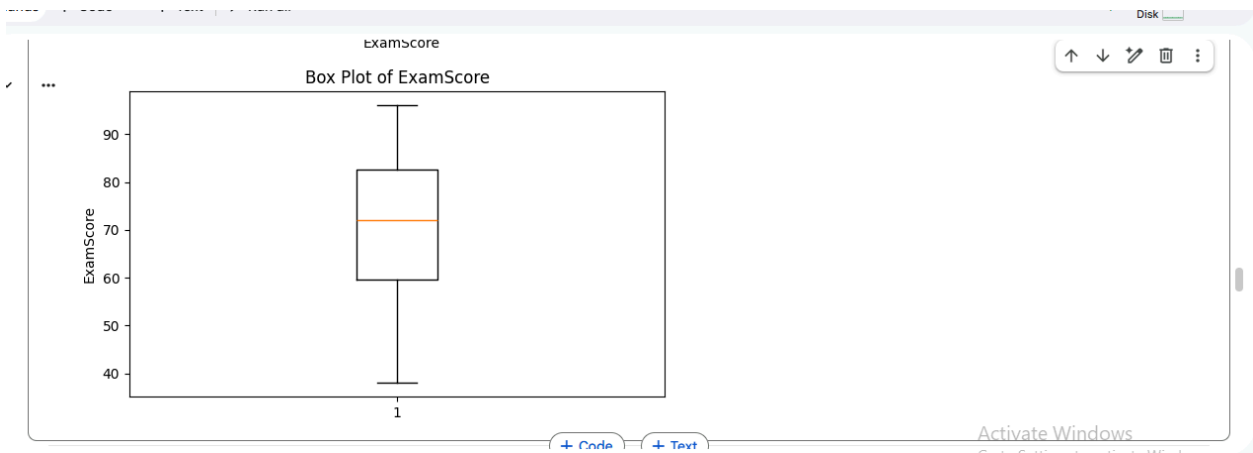
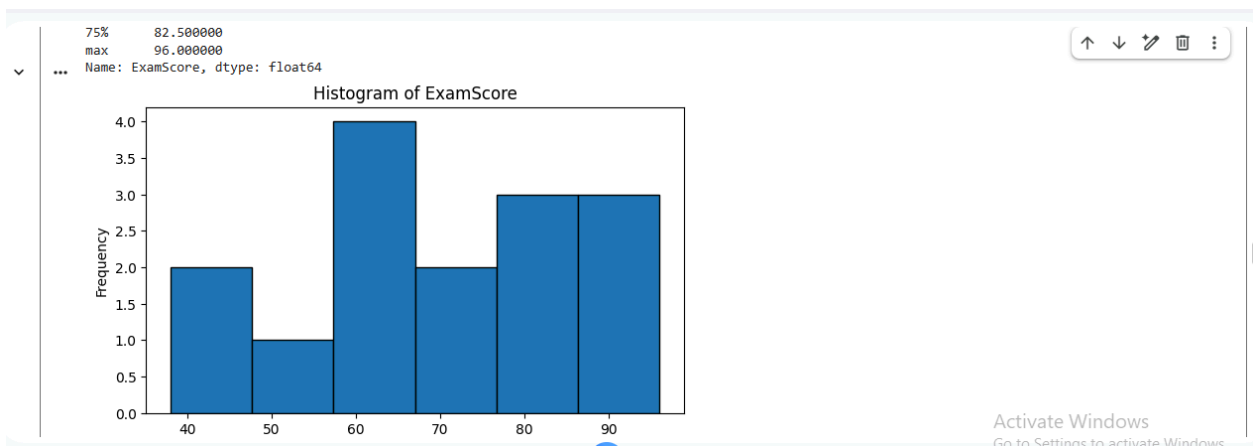
69.533333

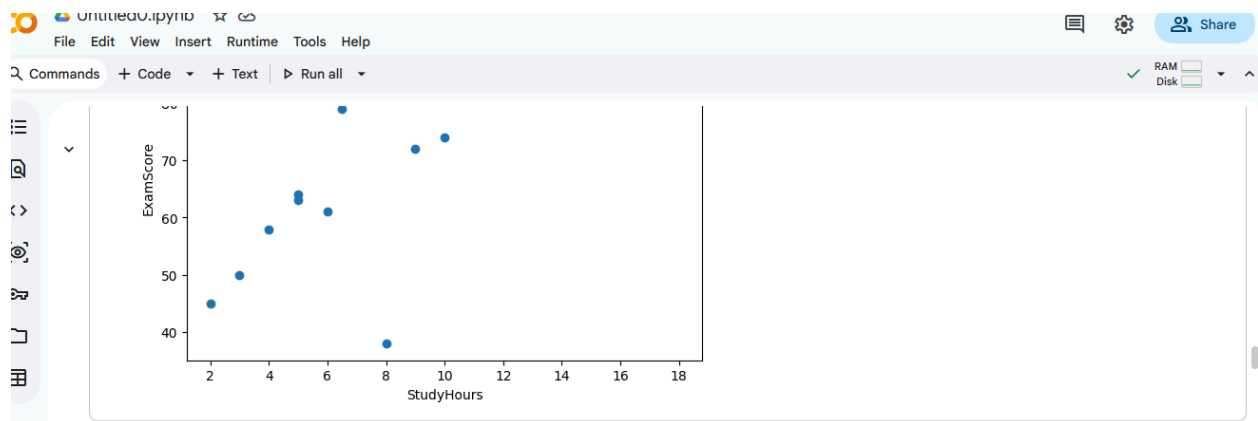
17.266262

38.000000

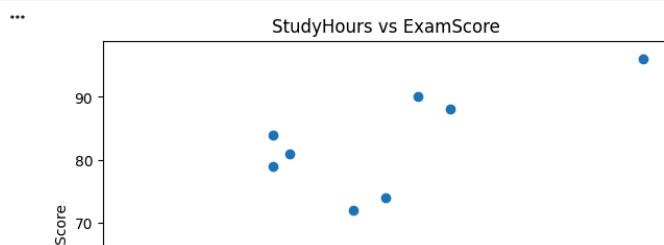
50.000000

Activate Windows

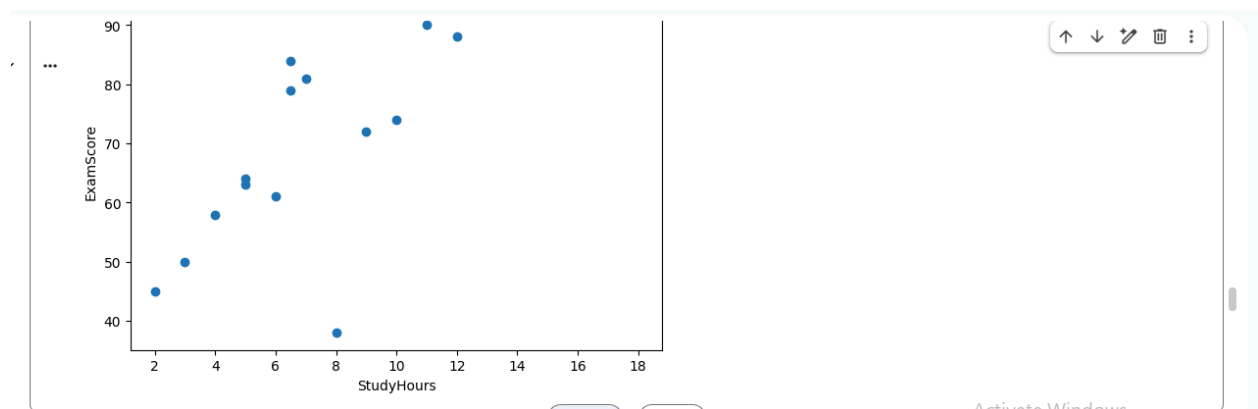


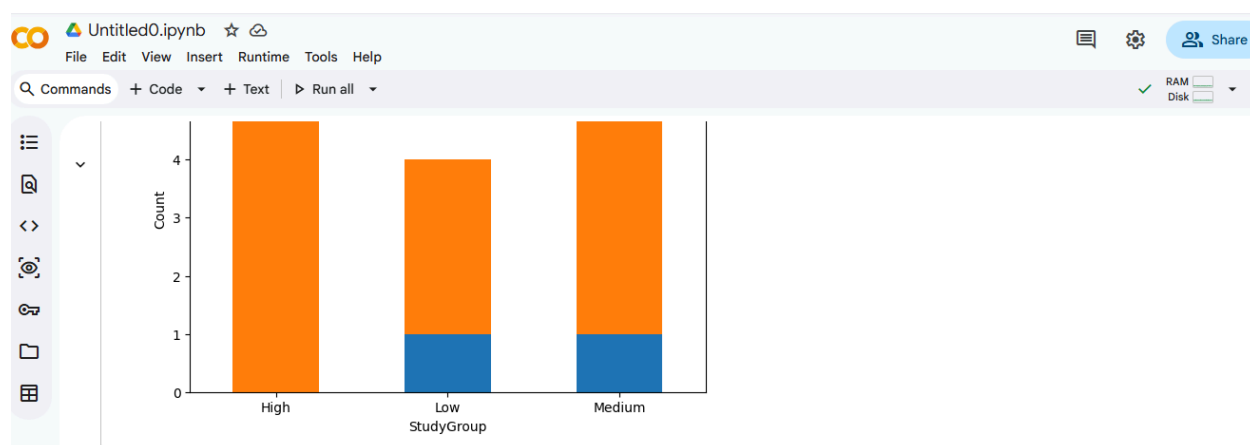


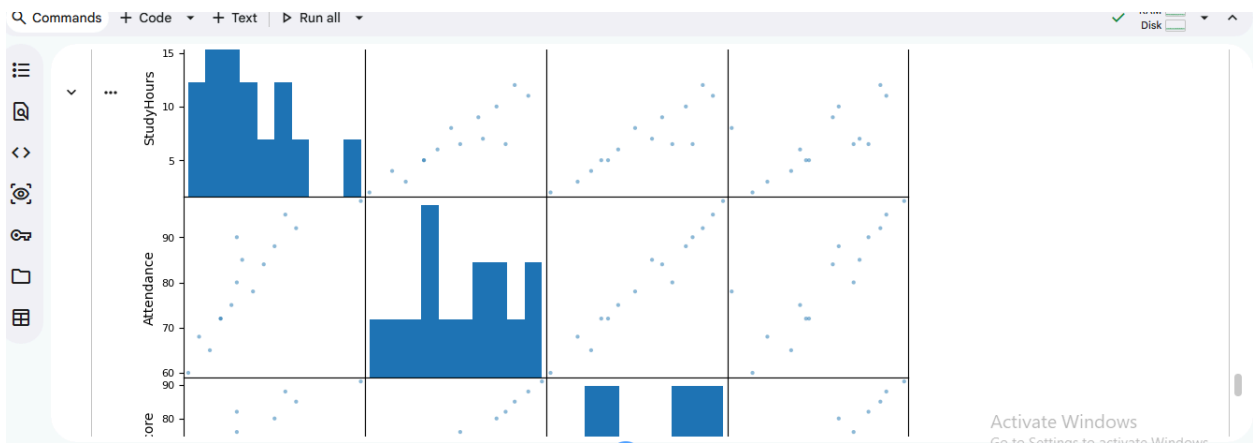
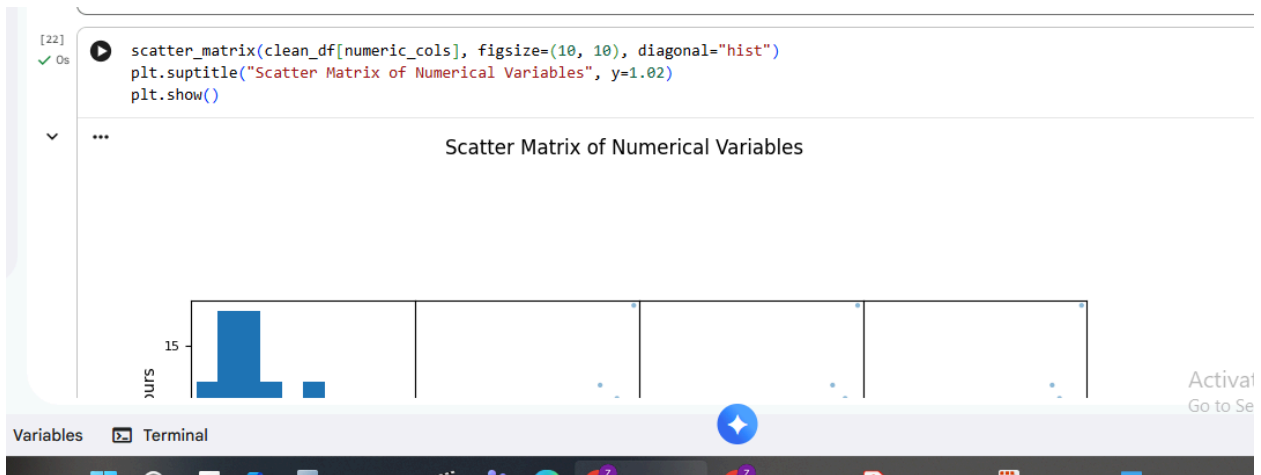
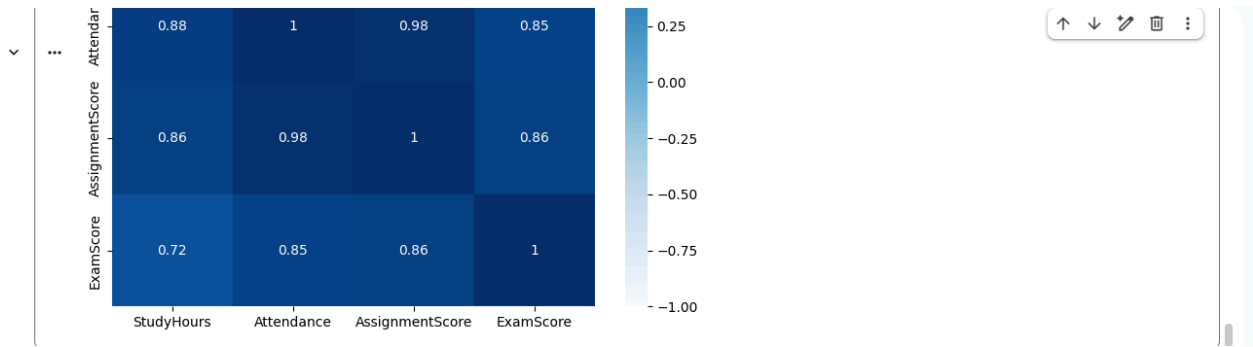
```
1 plt.figure(figsize=(7, 5))
2 plt.scatter(clean_df["StudyHours"], clean_df["ExamScore"])
3 plt.title("StudyHours vs ExamScore")
4 plt.xlabel("StudyHours")
5 plt.ylabel("ExamScore")
6 plt.show()
```

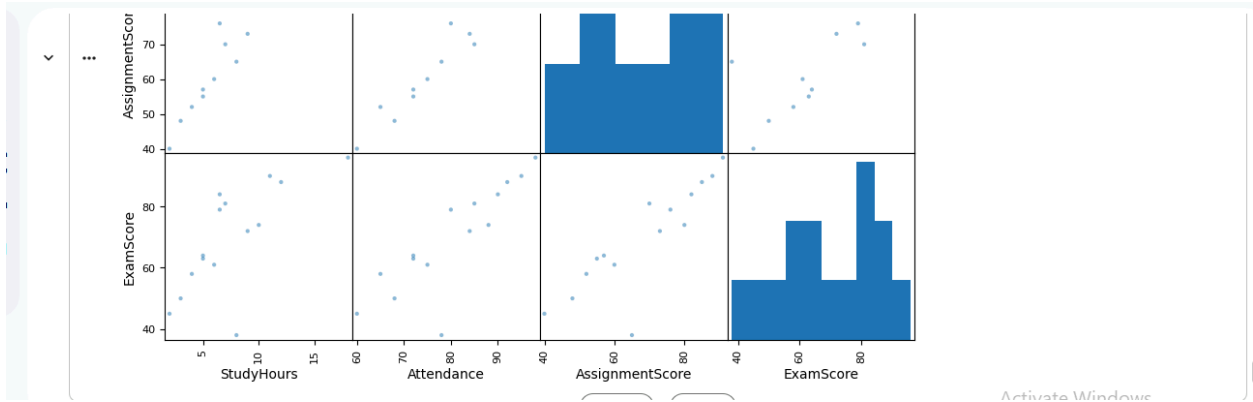


Activate Windows
Go to Settings to activate Windows.









17]
/ 0s

```
print("Example EDA findings:")
print("1. The cleaned dataset contains", clean_df.shape[0], "students and", clean_df.shape[1], "variables.")
print("2. Pass is more common than Fail in this dataset.")
print("3. Students with higher study hours generally tend to have higher exam scores.")
print("4. The High study group appears to have higher exam scores than the Low study group.")
print("5. ExamScore is positively correlated with StudyHours, Attendance, and AssignmentScore.")
```

▼

Example EDA findings:

1. The cleaned dataset contains 15 students and 7 variables.
2. Pass is more common than Fail in this dataset.
3. Students with higher study hours generally tend to have higher exam scores.
4. The High study group appears to have higher exam scores than the Low study group.
5. ExamScore is positively correlated with StudyHours, Attendance, and AssignmentScore.